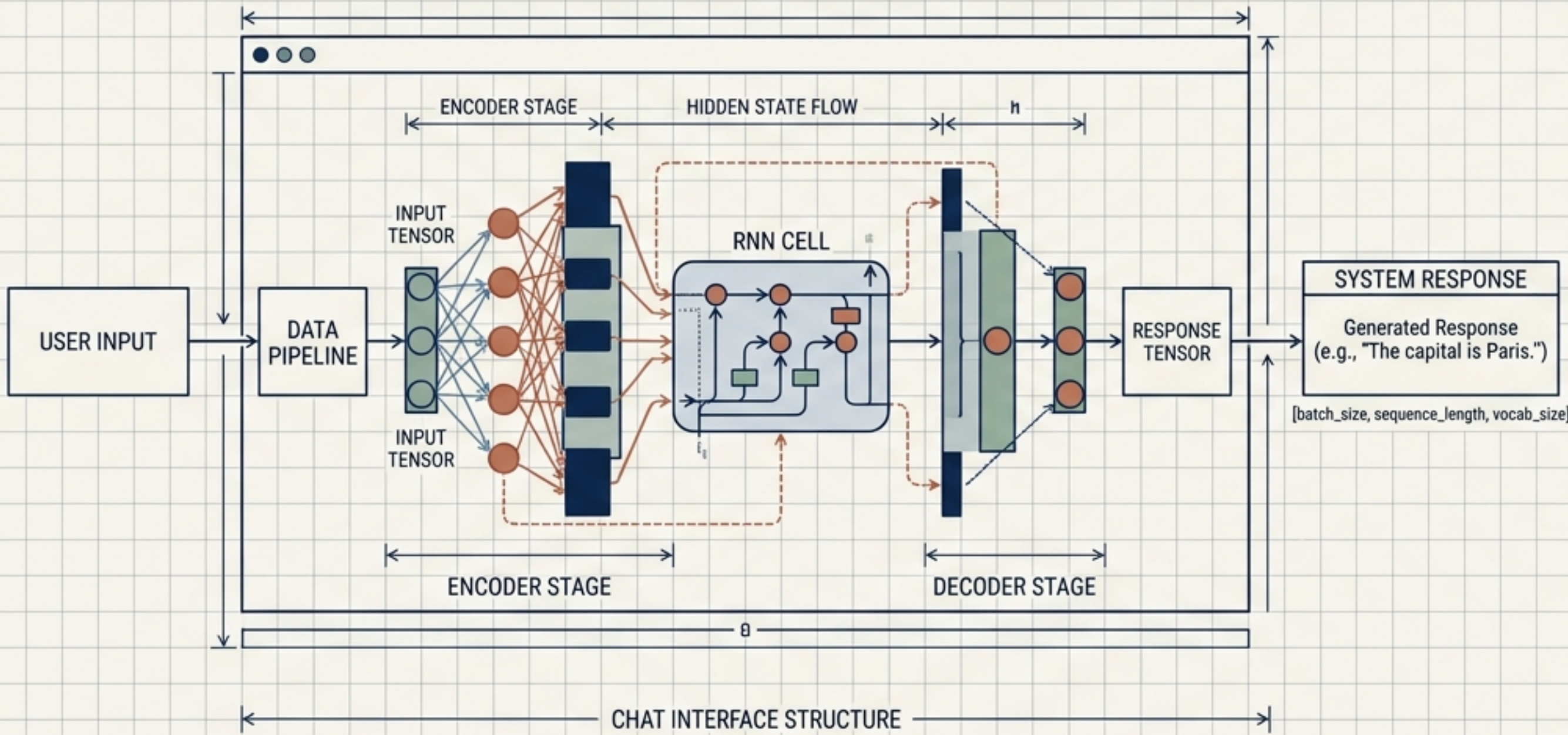
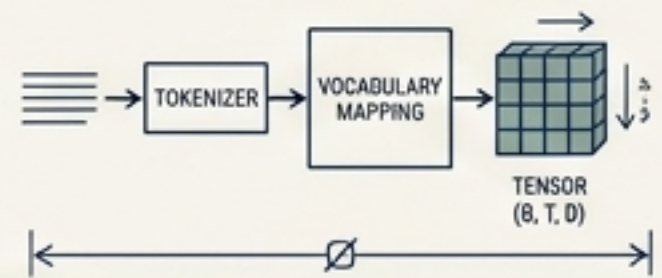


BUILDING A GENERAL KNOWLEDGE Q&A SYSTEM

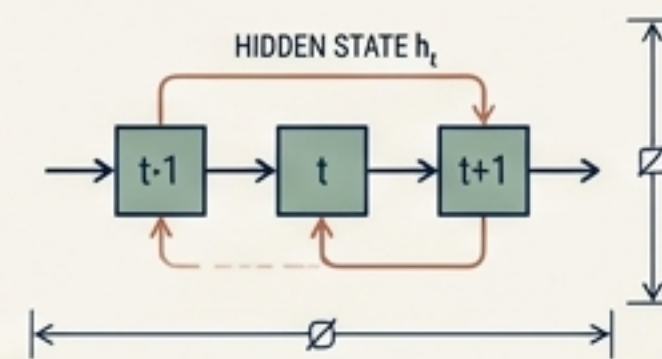
A PyTorch RNN Engineered From Scratch



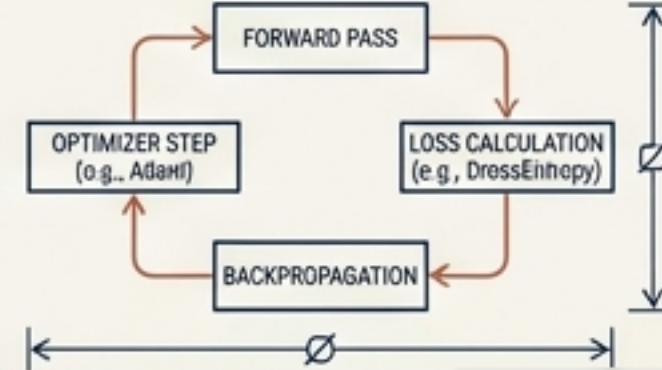
TEXT TO TENSORS: Engineering the data pipeline.



2. SEQUENTIAL MEMORY: Designing the Recurrent Neural Network.



3. THE TRAINING LOOP: Mastering PyTorch's learning mechanics.



The Reality of Conversational Data

The Messy Reality

Obstacle 1:
Conversational
Noise

Hey, what about this:
"What instrument measures pressure?"

What is the Capital
of 'France'?

What is the line
on xovernatos"?

Obstacle 2:
Punctuation Noise

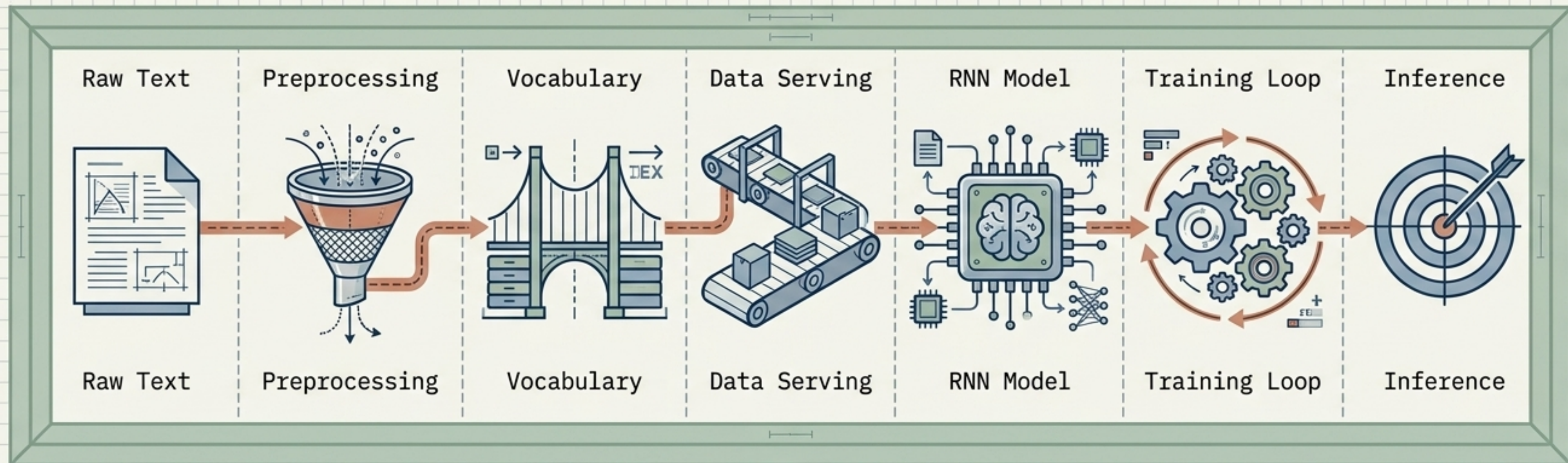
The Engineered Target

Target:
Single Word Prediction

Examples:
'Paris', 'barometer'

Instructor Note: To master fundamental RNN architecture without being overwhelmed by Sequence-to-Sequence padding complexities, we restrict our target to single-word precision.

The NLP Assembly Line



System Scope: From raw human string to machine intelligence.

Station 1: The Preprocessing Filter

What is the capital of 'France'?

Filter 1: Lowercase

`text.lower()`

what is the capital of 'france'?

Filter 2: Quote Removal

`re.sub(r'[""]', '', text)`

what is the capital of france?

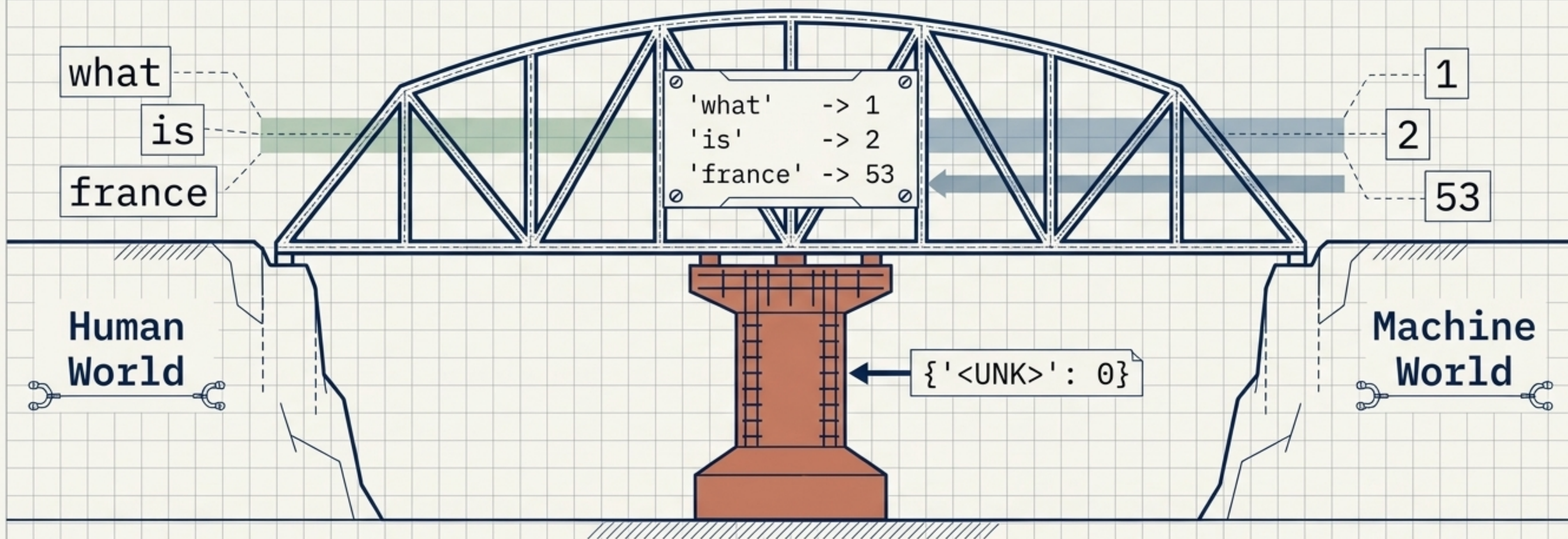
Filter 3:
Special Character Removal

`re.sub(r'^a-z0-9\s', ' ', text)`

The ^ inside the bracket means NOT.
It translates to:
"If it is NOT a lowercase letter,
number, or space, remove it!"

`['what', 'is', 'the', 'capital', 'of', 'france']`

Station 2: The Vocabulary Bridge

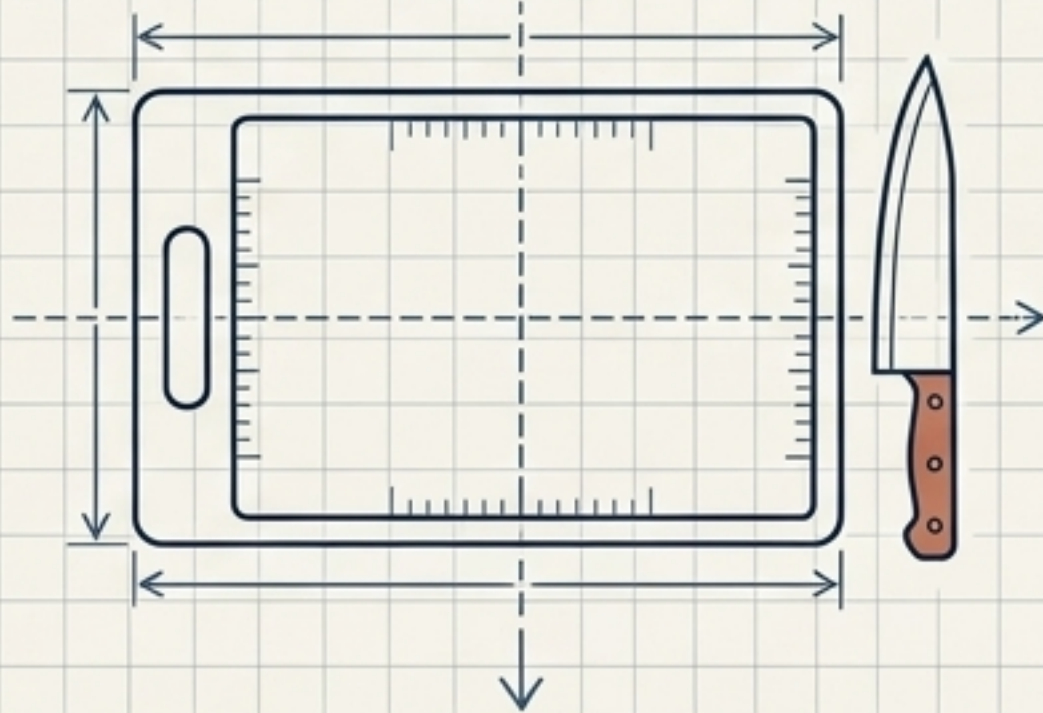


The Safety Net: What happens when our model sees a word it was never trained on? Without `<UNK>`, the system crashes. By assigning unknown words to Index 0, we build a safety net that allows the model to gracefully process unexpected input.

Data Handling: The Chef and the Waiter

The Chef

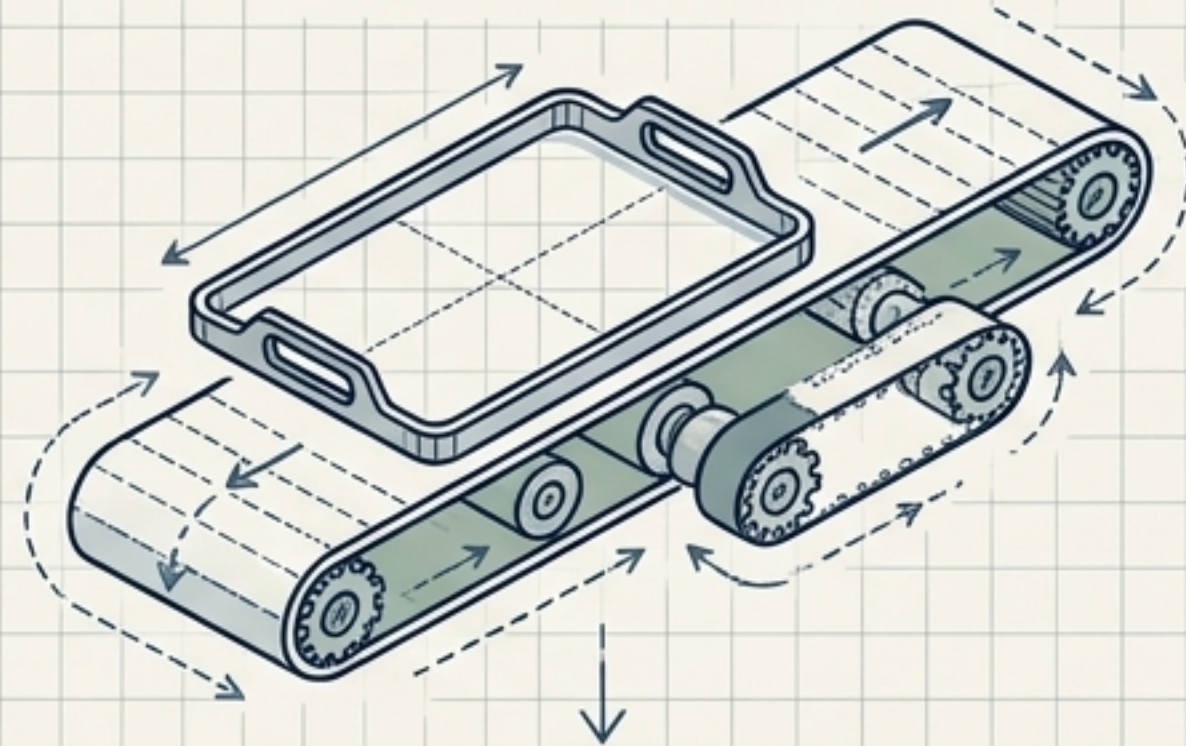
QADataset Class



- **Role:** Prepares the ingredients.
- **Mechanics:** Uses `__len__` (menu size) and `__getitem__` (prep on demand).
- **Action:** Converts raw vocabulary tokens into PyTorch Tensors.

The Waiter

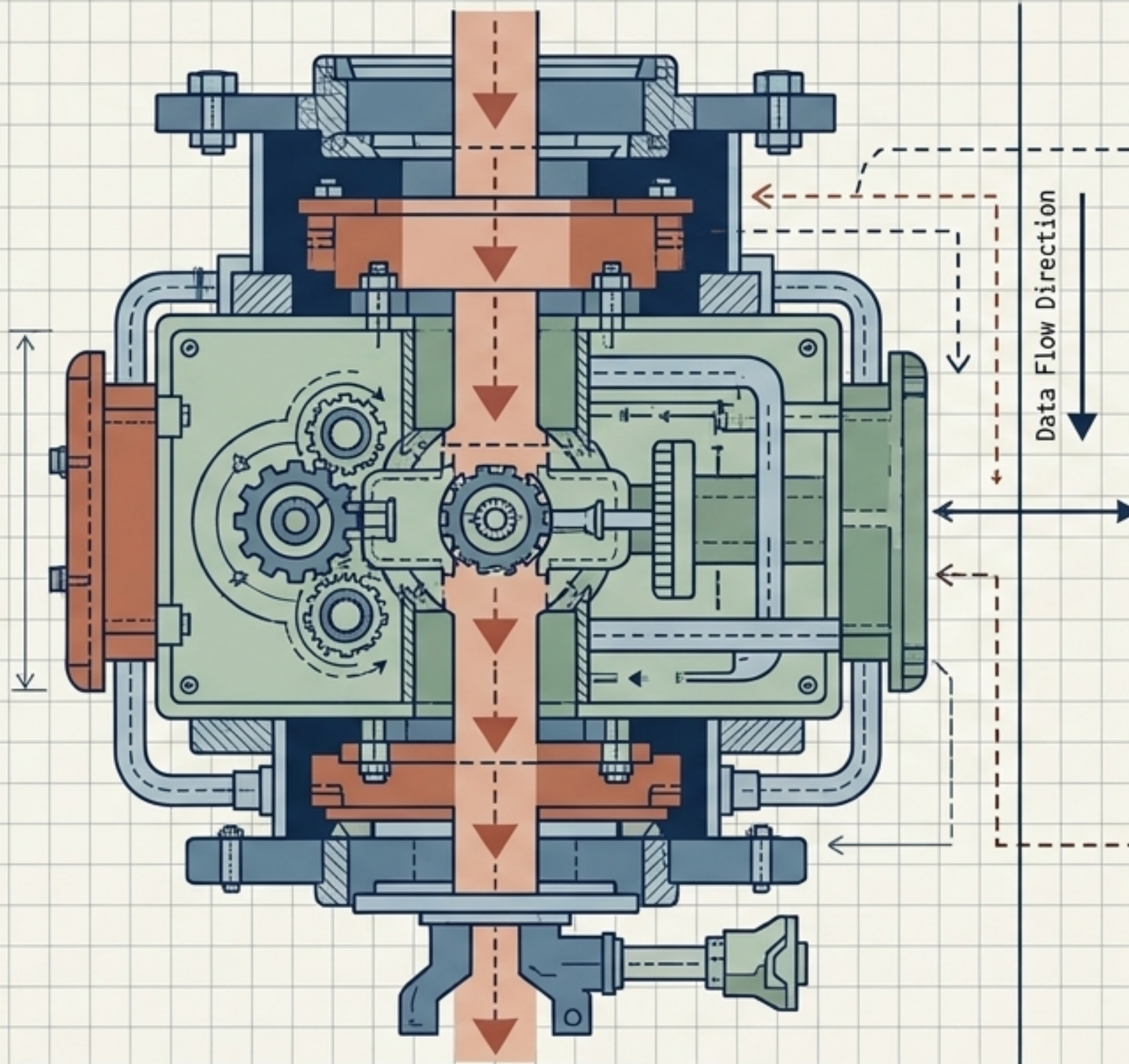
DataLoader Class



- **Role:** Serves the food to the model.
- **Mechanics:** Uses `batch_size=1` and `shuffle=True`.
- **Action:** Delivers tensors and randomizes the order so the model learns logic, not sequence.

Memory Efficiency Strategy: The Chef only prepares the specific tensors that the Waiter needs to serve in the current batch.

Station 3: Model Architecture Blueprint



Layer 1: Input & Comprehension

```
nn.Embedding(vocab_size, 50)
```

Transforms an isolated integer index into a dense 50-dimensional semantic vector.

Layer 2: Reading & Context (The Brain)

```
nn.RNN(50, 64, batch_first=True)
```

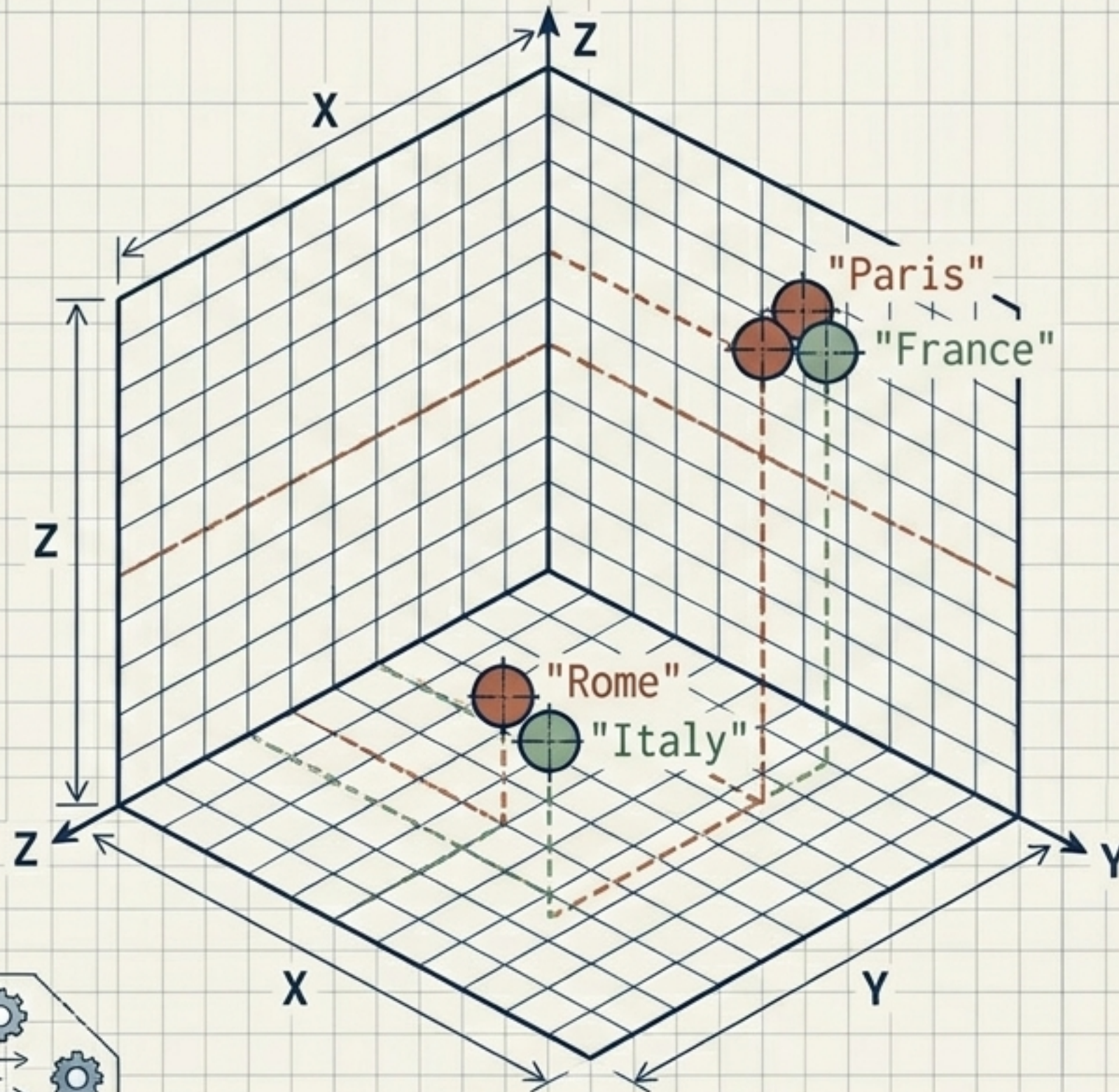
Processes the sequence step-by-step, maintaining a hidden state (memory) of 64 dimensions.

Layer 3: The Vocal Cord

```
nn.Linear(64, vocab_size)
```

Maps the 64-dimensional memory back into the total total vocabulary space to score every possible answer.

Deep Dive: The Embedding Layer



The Semantic Map

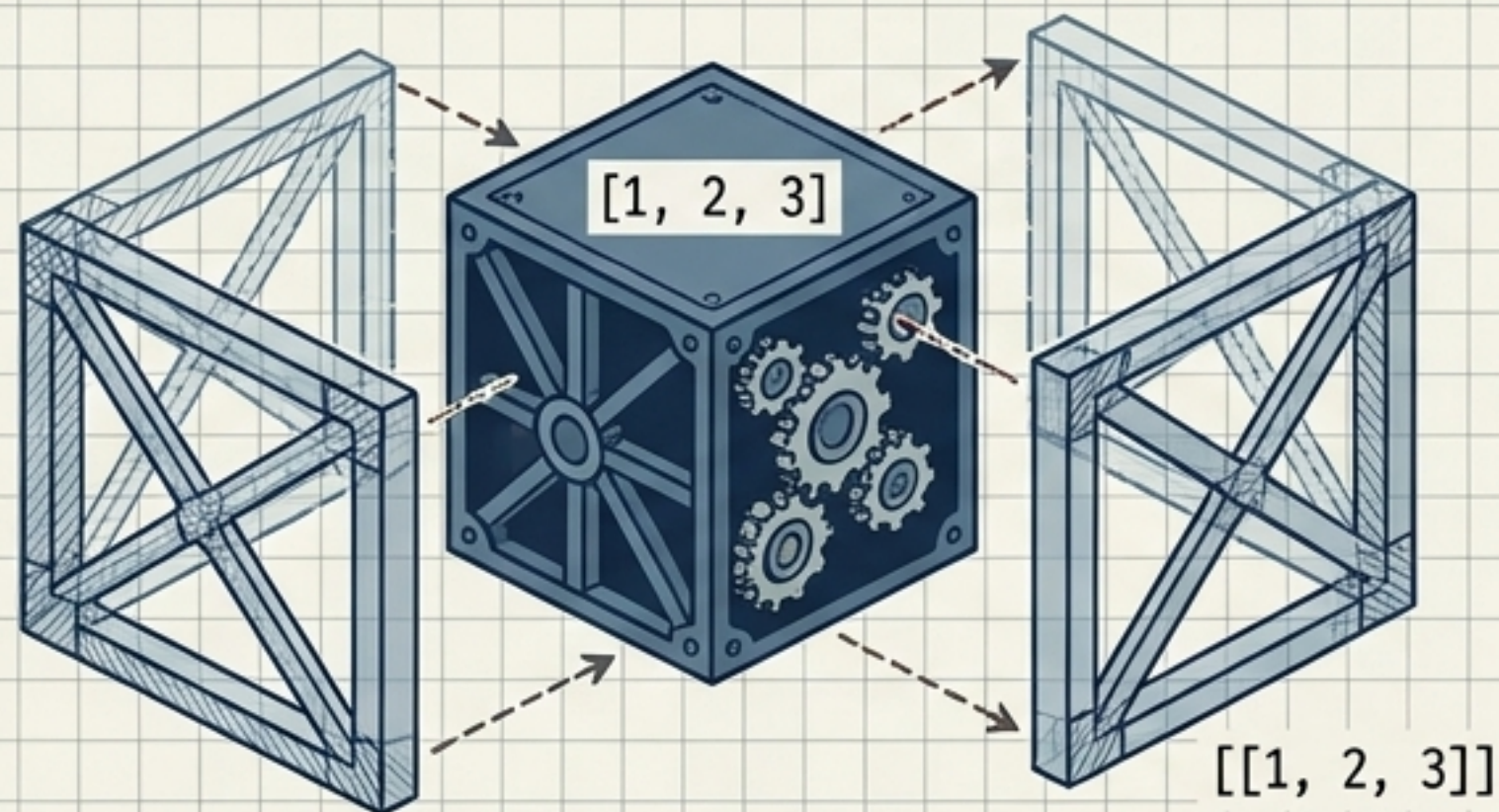
- Neural networks cannot calculate meaning from discrete integers (Index 53 vs Index 2).
- Embeddings map isolated integers into continuous dense vectors (e.g., 50 dimensions).
- **Result:** Words with shared contexts are physically positioned closer together in mathematical space.

Foundation Note: This spatial relationship architecture powers advanced systems like Word2Vec and modern Transformers.

Shape-Shifting: Mastering Tensor Dimensions

The Golden Rule: Count the brackets. `[[[1, 1, 3]]] = Shape (1, 1, 3)`

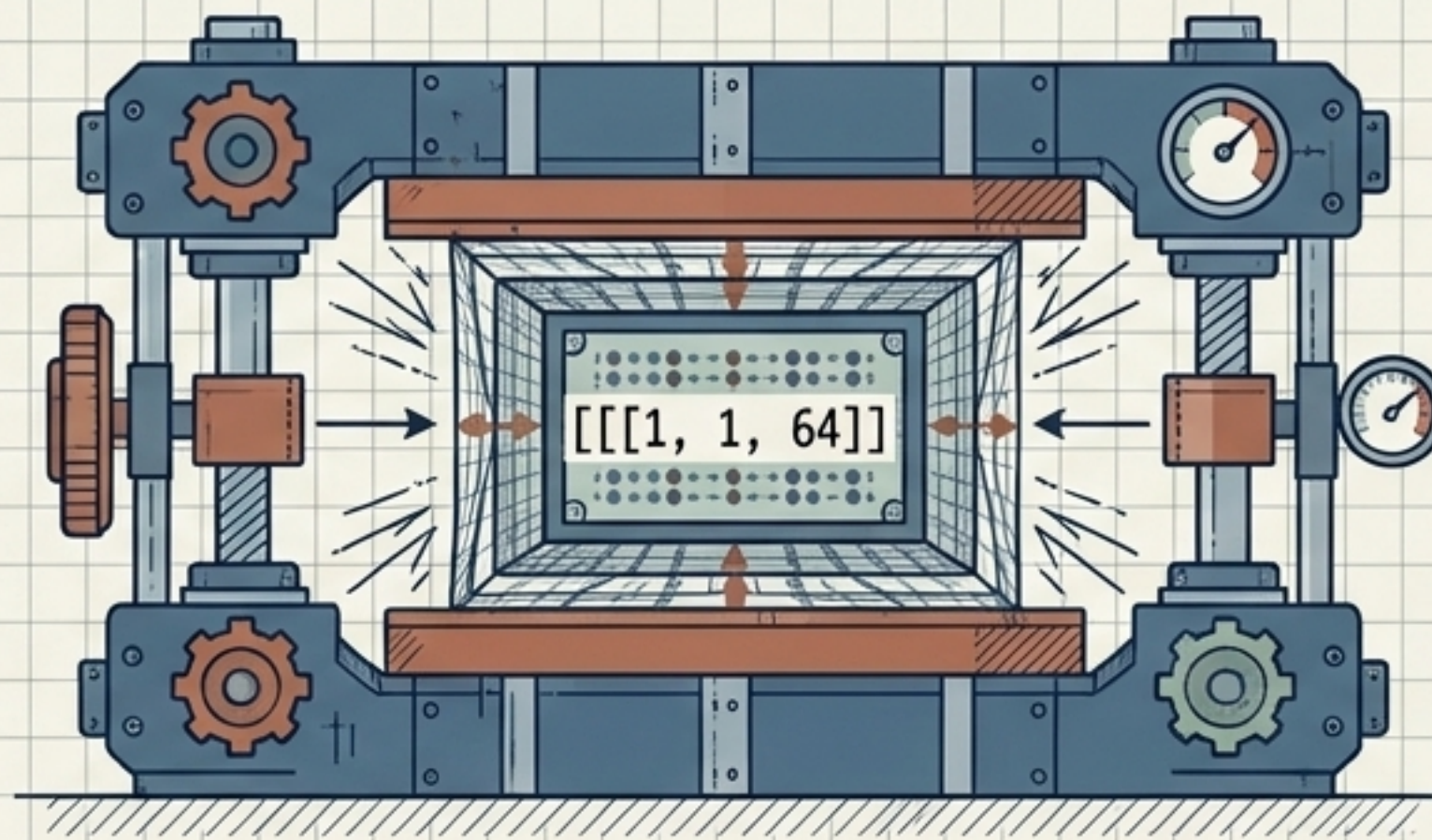
`unsqueeze(0)` - Adding a layer



Adds a dimension to satisfy PyTorch batch requirements.

Shape `(3) → (1, 3)`

`squeeze(0)` - Crushing a layer



Strips empty batch dimensions so the Linear layer can read the state.

Shape `(1, 1, 64) → (1, 64)`

RNN Tuple: `_, final = self.rnn(embedded)`

The `'_'` ignores intermediate steps; `'final'` captures the 64-dimension context.

The Training Loop: Classroom Dynamics

The Student

```
model(x)
```



Role: Takes the test. Generates raw predictions (logits) based on its current, imperfect understanding of the data.

The Examiner

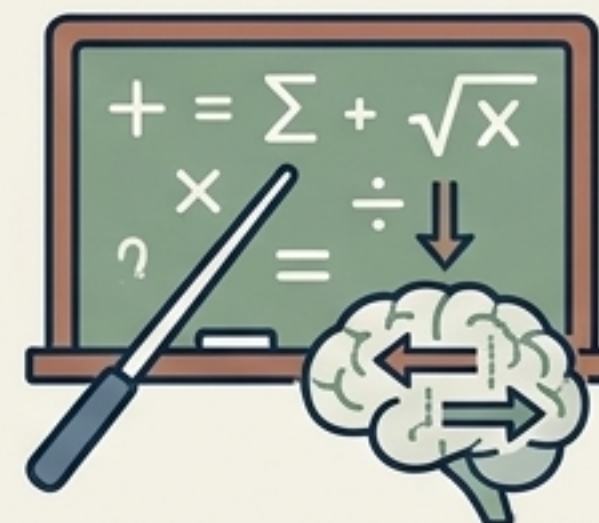
```
nn.CrossEntropyLoss()
```



Role: Grades the test. Compares the Student's prediction against the target answer and calculates the precise mathematical error.

The Teacher

```
torch.optim.Adam(lr=0.0001)
```

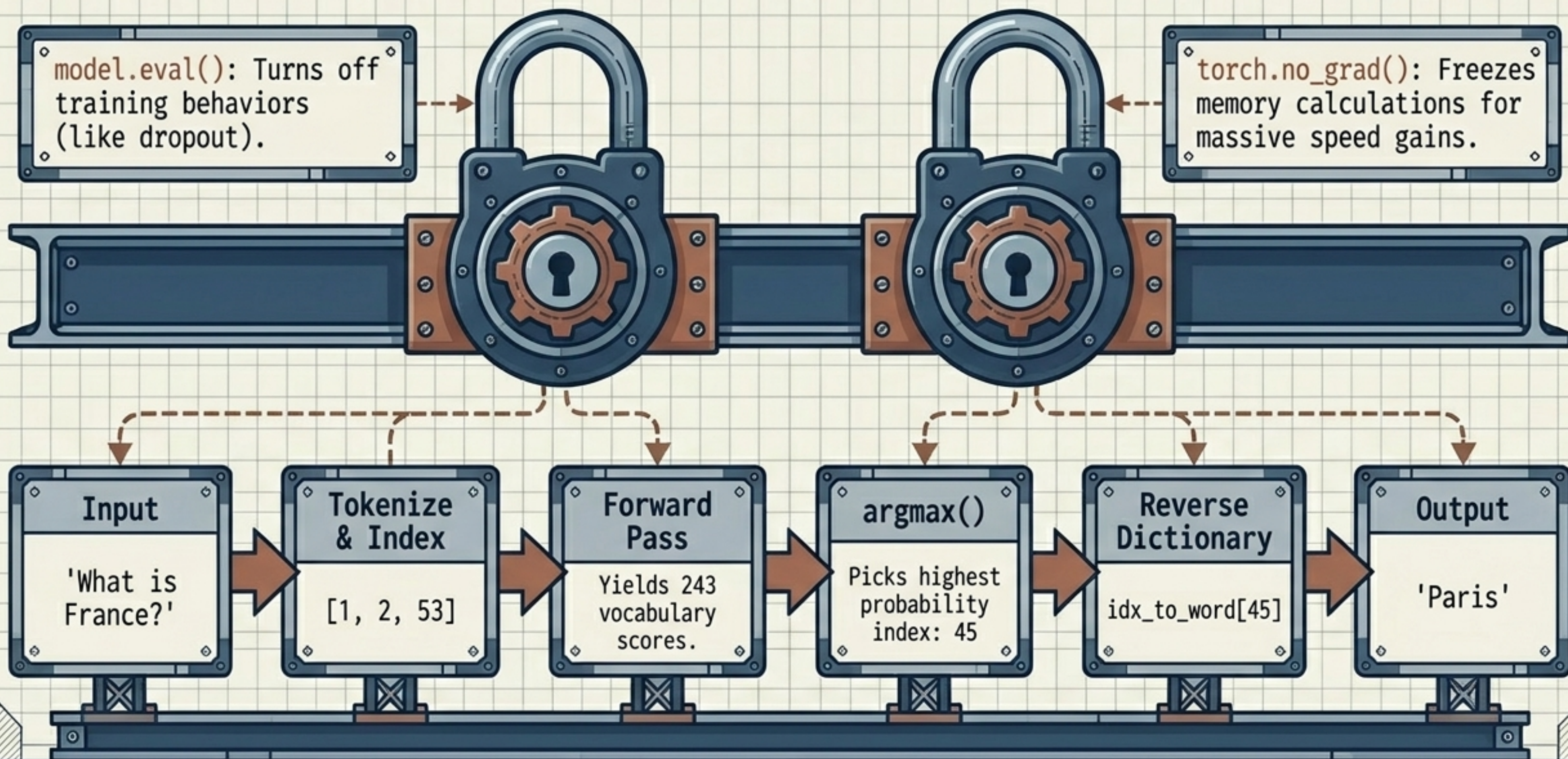


Role: Guides improvement. Adjusts the Student's internal weights based on the Examiner's grade. Learning rate (lr) dictates how drastically to change the brain.

The 5 Golden Steps of PyTorch Training



Station 4: Inference & Prediction



Synthesis: The Data Shape Journey

Phase	Data Format	PyTorch Tensor Shape	Concrete Example
Raw Text	String	N/A	'France'
Tokenization	Integer List	[1]	[53]
Embedding Layer	Dense Vector	(1, 1, 50)	[0.2, -0.1, 0.8...]
RNN Core (Final State)	Context Memory	(1, 64)	Contextual summary of the query
Linear Layer	Logits / Scores	(1, Vocab_Size)	Raw scores for all known words
Argmax Function	Highest Probability	String	'Paris'

The Complete Pipeline: From human inquiry to machine intelligence.